Guía Metodológica
Problema 1: Sistema de Creación de Personajes de Videojuegos

Ingeniería de Software

Índice

1. **Introducción** **2**
2. **Paso 0: Leer y comprender el problema** **2**
3. **Paso 1: Identificar los conceptos del dominio** **3**
4. **Paso 2: Reconocer el problema de diseño** **3**
5. **Paso 3: Buscar un patrón adecuado** **4**
6. **Paso 4: Diseñar la estructura de clases** **5**
7. **Paso 5: Implementar los productos abstractos** **6**
8. **Paso 6: Implementar los productos concretos** **7**
9. **Paso 7: Implementar la fábrica abstracta** **9**
10. **Paso 8: Implementar las fábricas concretas** **9**
11. **Paso 9: Implementar el cliente** **10**
12. **Paso 10: Probar la solución** **11**
13. **Paso 11: Analizar la extensibilidad** **12**
14. **Paso 12: Integrar con otros patrones (inciso d)** **13**
15. **Paso 13: Calcular el costo en clases (inciso e)** **14**
16. **Paso 14: Restricción final - Carga dinámica** **14**
17. **Resumen del proceso** **15**
18. **Ejercicios propuestos** **16**

1. Introducción

Esta guía te acompañará en la resolución del **Problema 1: Sistema de Creación de Personajes de Videojuegos**, implementado en **C#**. No te daremos la solución directamente: iremos construyéndola juntos, paso a paso, a través de preguntas, pistas y razonamientos.

El objetivo es que aprendas a *pensar* como un diseñador de software: identificar problemas, reconocer patrones, tomar decisiones y justificarlas.

Cuidado

A lo largo de la guía encontrarás cajas de colores:

- **Azul:** preguntas que debes hacerte antes de avanzar.
- **Verde:** razonamientos que te ayudan a responder.
- **Amarillo:** pistas concretas si te quedas atascado.
- **Rosa:** errores comunes que debes evitar.

2. Paso 0: Leer y comprender el problema

Pregunta guía

Lee atentamente el enunciado:
"Una aplicación de videojuegos necesita crear personajes y equipamiento que sean visualmente consistentes con diferentes eras históricas (Edad Media, Era Futurista). Cada era tiene sus propias variantes de armas y armaduras que deben coordinarse. No se pueden mezclar elementos de diferentes eras en un mismo personaje."

Antes de escribir una sola línea de código, necesitas entender **qué se pide**. Responde mentalmente:

1. ¿Cuáles son los **objetos** que se crean? (personajes, armas, armaduras...)
2. ¿Cuáles son las **variantes** de esos objetos? (eras históricas)
3. ¿Cuál es la **restricción principal**? (no mezclar eras)
4. ¿Qué **necesidades futuras** se vislumbran? (nuevas eras, nuevos productos)

Razonamiento

Al leer el problema, identifica:

- **Familias de productos:** cada era produce su propio conjunto de objetos (armas, armaduras, monturas, habilidades).
- **Productos dentro de una familia:** dentro de la era Medieval, hay espada, armadura de placas, caballo, fuerza bruta.
- **Restricción de consistencia:** un personaje medieval *no* puede tener una pistola láser.
- **Necesidad de extensibilidad:** mañana podrían añadir una era "Post-apocalíptica".

Conclusión: el problema no es solo "crear objetos", sino *crear familias de objetos que encajen entre sí.*

3. Paso 1: Identificar los conceptos del dominio

Pregunta guía

Antes de pensar en patrones, **¿qué conceptos existen en el dominio del problema?** Haz una lista de sustantivos relevantes.

Lista tentativa:

- **Personaje** (el cliente que usa los objetos)
- **Era** (Medieval, Futurista, Antigua)
- **Arma**, **Armadura**, **Montura**, **Habilidad** (los tipos de producto)
- **Variantes concretas:** Espada, PistolaLaser, Lanza, etc.

Pista

No te preocupes todavía por clases ni interfaces. Solo identifica *qué cosas existen* en el problema. La estructura vendrá después.

4. Paso 2: Reconocer el problema de diseño

Pregunta guía

Si tuvieras que crear estos objetos sin ningún patrón, **¿cómo lo harías?** Piensa en una solución "ingenua".

Solución ingenua (con `if/else` o `switch`):

```
public class Personaje
{
    private string era;
    private object arma;
    private object armadura;

    public Personaje(string era)
    {
        this.era = era;
        if (era == "medieval")
        {
            arma = new Espada();
            armadura = new ArmaduraPlacas();
        }
        else if (era == "futurista")
        {
            arma = new PistolaLaser();
            armadura = new Exoesqueleto();
        }
        else if (era == "antigua")
        {
            arma = new Lanza();
            armadura = new ArmaduraCuero();
```

```
                }
            }
        }

```

Listing 1: Solución ingenua con condicionales

Pregunta guía

¿Qué problemas tiene esta solución? Piensa en:

- Añadir una nueva era.
- Añadir un nuevo tipo de producto (por ejemplo, “Montura”).
- Evitar mezclar eras por error.
- Reutilizar el código en otro lugar.

Razonamiento

Problemas de la solución ingenua:

1. **Acoplamiento fuerte:** `Personaje` conoce todas las clases concretas.
2. **Crecimiento explosivo:** cada nueva era añade más ramas al `if`.
3. **Fácil de romper:** nada impide crear un personaje medieval con una pistola láser.
4. **Difícil de extender:** añadir un nuevo producto (Montura) obliga a tocar todos los `if`.
5. **Violación de encapsulamiento:** la lógica de creación está dentro del cliente.

Necesitamos una solución que:

- Aísle la creación de objetos.
- Garantice la consistencia entre productos de una misma familia.
- Permita añadir nuevas familias sin tocar el código existente.
- Permita añadir nuevos productos sin reescribir todo.

5. Paso 3: Buscar un patrón adecuado

Pregunta guía

¿Conoces algún patrón de diseño que resuelva el problema de **crear familias de objetos relacionados**? Piensa en los patrones creacionales.

Patrones creacionales que podrías considerar:

Patrón	¿Sirve para este problema?
Singleton	No: garantiza una única instancia, pero no crea familias.
Factory Method	Parcial: crea un producto, pero no coordina familias.
Abstract Factory	**Sí:** crea familias de productos relacionados.
Builder	No: construye un objeto complejo paso a paso, no familias.
Prototype	No: clona objetos, pero no garantiza consistencia entre familias.

Pista

Cuando el problema menciona *"familias de productos que deben coordinarse"* y *"no mezclar elementos"*, el patrón que suena es **Abstract Factory**.

Razonamiento

Abstract Factory es adecuado porque:

- Define una **interfaz** para crear familias de productos.
- Cada **fábrica concreta** crea productos de una sola familia (una era).
- El cliente usa la fábrica abstracta y no conoce las clases concretas.
- Añadir una nueva era = añadir una nueva fábrica concreta, sin tocar el cliente.
- La consistencia está garantizada: una fábrica medieval *solo* crea productos medievales.

6. Paso 4: Diseñar la estructura de clases

Pregunta guía

Ahora que sabes que usarás Abstract Factory, **¿qué clases necesitas?** Piensa en:

- Las interfaces de los productos.
- Las clases concretas de los productos.
- La interfaz de la fábrica abstracta.
- Las fábricas concretas.
- El cliente.

Estructura propuesta:

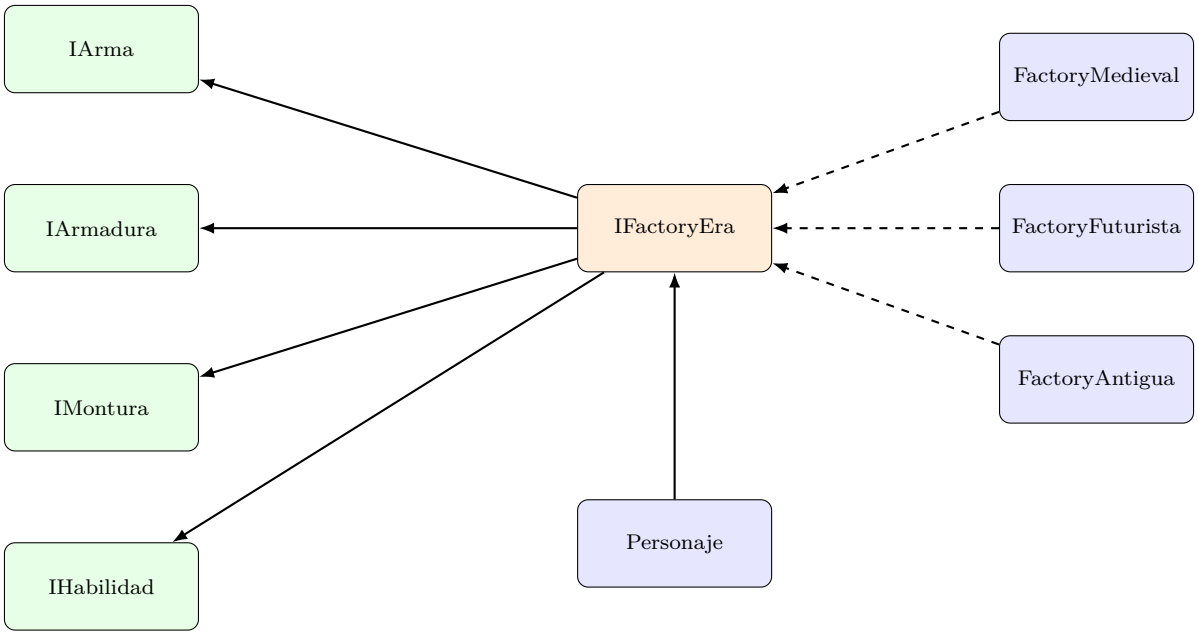
1. **Productos abstractos:** `IArma`, `IArmadura`, `IMontura`, `IHabilidad`.
2. **Productos concretos:** `Espada`, `PistolaLaser`, `Lanza`, etc.
3. **Fábrica abstracta:** `IFactoryEra` con métodos `CrearArma()`, `CrearArmadura()`, etc.
4. **Fábricas concretas:** `FactoryMedieval`, `FactoryFuturista`, `FactoryAntigua`.

5. **Cliente:** `Personaje`, que recibe una `IFactoryEra`.

Cuidado

En C#, por convención, las interfaces empiezan con `I` mayúscula: `IArma`, `IFactoryEra`, etc. Esto es una diferencia importante con Java.

Diagrama de clases



Cuidado

No te preocupes si el diagrama no es perfecto al principio. Lo importante es que veas la **relación**: el cliente depende de la fábrica abstracta, no de las concretas.

7. Paso 5: Implementar los productos abstractos

Pregunta guía

¿Qué métodos deberían tener las interfaces de los productos? Piensa en qué *acciones* realizan.

```
        public interface IArma
        {
            void Usar();
        }

        public interface IArmadura
        {
            void Equipar();
        }

        public interface IMontura
        {
            void Montar();
        }

```

```
16          public interface IHabilidad
17          {
18              void Activar();
19          }
20
```

Listing 2: Productos abstractos (interfaces)

Razonamiento

Cada interfaz define el **contrato** que todos los productos de ese tipo deben cumplir. El cliente solo conocerá estas interfaces, nunca las clases concretas.
Observa que en C# los métodos de las interfaces se escriben en **PascalCase** (`Usar`, no `usar`), por convención del lenguaje.

8. Paso 6: Implementar los productos concretos

Pregunta guía

Ahora implementa los productos concretos para **cada era**. ¿Cuántas clases necesitas?

Para 3 eras y 4 productos: $3 \times 4 = 12$ clases concretas.

```
1           public class Espada : IArma
2           {
3               public void Usar()
4               {
5                   Console.WriteLine("Atacando con espada");
6               }
7           }
8
9           public class ArmaduraPlacas : IArmadura
10          {
11              public void Equipar()
12              {
13                  Console.WriteLine("Equipando armadura de placas");
14              }
15          }
16
17          public class Caballo : IMontura
18          {
19              public void Montar()
20              {
21                  Console.WriteLine("Montando caballo");
22              }
23          }
24
25          public class FuerzaBruta : IHabilidad
26          {
27              public void Activar()
28              {
29                  Console.WriteLine("Activando fuerza bruta");
30              }
31          }
32
```

Listing 3: Productos concretos - Era Medieval

```
1           public class PistolaLaser : IArma
2           {
```

```
            public void Usar()
            {
                Console.WriteLine("Disparando laser");
            }
        }

        public class Exoesqueleto : IArmadura
        {
            public void Equipar()
            {
                Console.WriteLine("Equipando exoesqueleto");
            }
        }

        public class MotoHover : IMontura
        {
            public void Montar()
            {
                Console.WriteLine("Montando moto hover");
            }
        }

        public class Hackeo : IHabilidad
        {
            public void Activar()
            {
                Console.WriteLine("Hackeando sistemas");
            }
        }

```

Listing 4: Productos concretos - Era Futurista

```
        public class Lanza : IArma
        {
            public void Usar()
            {
                Console.WriteLine("Atacando con lanza");
            }
        }

        public class ArmaduraCuero : IArmadura
        {
            public void Equipar()
            {
                Console.WriteLine("Equipando armadura de cuero");
            }
        }

        public class Elefante : IMontura
        {
            public void Montar()
            {
                Console.WriteLine("Montando elefante");
            }
        }

        public class Furia : IHabilidad
        {
            public void Activar()
            {
                Console.WriteLine("Activando furia");
            }
        }
```

```
32
```

Listing 5: Productos concretos - Era Antigua

Pista

Observa que cada clase concreta implementa *una* interfaz de producto. En C# se usa `:` (dos puntos) para indicar implementación de interfaz, no `implements` como en Java.
No hay herencia entre productos de diferentes eras: son independientes.

9. Paso 7: Implementar la fábrica abstracta

Pregunta guía

¿Qué métodos debe tener la fábrica abstracta? Piensa: ¿qué productos necesita crear el cliente?

```
1       public interface IFactoryEra
2       {
3           IArma CrearArma();
4           IArmadura CrearArmadura();
5           IMontura CrearMontura();
6           IHabilidad CrearHabilidad();
7       }
8
```

Listing 6: Fábrica abstracta

Razonamiento

La fábrica abstracta declara **un método por cada tipo de producto**. Cada fábrica concreta implementará estos métodos devolviendo los productos de su era.
En C#, los métodos se nombran en PascalCase: `CrearArma()`, no `crearArma()`.

10. Paso 8: Implementar las fábricas concretas

Pregunta guía

Ahora implementa una fábrica por cada era. ¿Qué debe devolver cada método?

```
1       public class FactoryMedieval : IFactoryEra
2       {
3           public IArma CrearArma() { return new Espada(); }
4           public IArmadura CrearArmadura() { return new ArmaduraPlacas(); }
5           public IMontura CrearMontura() { return new Caballo(); }
6           public IHabilidad CrearHabilidad() { return new FuerzaBruta(); }
7       }
8
9       public class FactoryFuturista : IFactoryEra
10      {
11          public IArma CrearArma() { return new PistolaLaser(); }
12          public IArmadura CrearArmadura() { return new Exoesqueleto(); }
13          public IMontura CrearMontura() { return new MotoHover(); }
14          public IHabilidad CrearHabilidad() { return new Hackeo(); }
15      }
```

```
16
17      public class FactoryAntigua : IFactoryEra
18      {
19          public IArma CrearArma() { return new Lanza(); }
20          public IArmadura CrearArmadura() { return new ArmaduraCuero(); }
21          public IMontura CrearMontura() { return new Elefante(); }
22          public IHabilidad CrearHabilidad() { return new Furia(); }
23      }
24
```

Listing 7: Fábricas concretas

Cuidado

Cada fábrica concreta **solo** crea productos de su era. Esto es lo que garantiza que no se mezclen. Si por error `FactoryMedieval` devolviera una `PistolaLaser`, romperíamos la consistencia.

11. Paso 9: Implementar el cliente

Pregunta guía

¿Cómo debe usar el `Personaje` la fábrica? ¿Debe conocer las clases concretas?

```
1       public class Personaje
2       {
3           private IArma arma;
4           private IArmadura armadura;
5           private IMontura montura;
6           private IHabilidad habilidad;
7
8           public Personaje(IFactoryEra factory)
9           {
10              this.arma = factory.CrearArma();
11              this.armadura = factory.CrearArmadura();
12              this.montura = factory.CrearMontura();
13              this.habilidad = factory.CrearHabilidad();
14          }
15
16          public void Mostrar()
17          {
18              arma.Usar();
19              armadura.Equipar();
20              montura.Montar();
21              habilidad.Activar();
22          }
23      }
24
```

Listing 8: Cliente Personaje

Razonamiento

El `Personaje` **solo conoce** la interfaz `IFactoryEra` y las interfaces de los productos. No sabe si el arma es una espada o una pistola láser. Esto es lo que hace que el código sea flexible.

12. Paso 10: Probar la solución

Pregunta guía

¿Cómo probarías que la solución funciona? Escribe un pequeño programa que cree un personaje de cada era.

```
        public class Program
        {
            public static void Main(string[] args)
            {
                Console.WriteLine("=== Personaje Medieval ===");
                Personaje medieval = new Personaje(new FactoryMedieval());
                medieval.Mostrar();

                Console.WriteLine("\n=== Personaje Futurista ===");
                Personaje futurista = new Personaje(new FactoryFuturista());
                futurista.Mostrar();

                Console.WriteLine("\n=== Personaje Antiguo ===");
                Personaje antiguo = new Personaje(new FactoryAntigua());
                antiguo.Mostrar();
            }
        }

```

Listing 9: Prueba de la solución

Salida esperada:

```
=== Personaje Medieval ===
Atacando con espada
Equipando armadura de placas
Montando caballo
Activando fuerza bruta

=== Personaje Futurista ===
Disparando laser
Equipando exoesqueleto
Montando moto hover
Hackeando sistemas

=== Personaje Antiguo ===
Atacando con lanza
Equipando armadura de cuero
Montando elefante
Activando furia
```

Conclusión

Si la salida es correcta, has implementado el patrón **Abstract Factory** correctamente. El cliente no conoce las clases concretas y la consistencia entre productos está garantizada.

13. Paso 11: Analizar la extensibilidad

Pregunta guía

Imagina que mañana quieren añadir una era **Post-apocalíptica**. ¿Qué tendrías que cambiar?

Razonamiento

Solo necesitas:

1. Crear 4 nuevas clases de productos concretos: `BateClavos`, `ArmaduraChatarra`, `MotoOxidada`, `Supervivencia`.
2. Crear una nueva fábrica: `FactoryPostApocaliptica`.
3. **No tocar** `Personaje`, ni `IFactoryEra`, ni las fábricas existentes.

Esto demuestra que el diseño es **extensible**: puedes añadir nuevas eras sin modificar el código existente.

Pregunta guía

¿Y si quieres añadir un nuevo tipo de producto, por ejemplo `IEscudo`? ¿Qué tendrías que cambiar?

Razonamiento

Aquí la historia es diferente:

1. Crear la interfaz `IEscudo`.
2. Crear `EscudoMedieval`, `EscudoFuturista`, `EscudoAntiguo`.
3. **Modificar** `IFactoryEra` para añadir `CrearEscudo()`.
4. **Modificar** todas las fábricas concretas para implementar el nuevo método.
5. **Modificar** `Personaje` si quiere usar el escudo.

Esto muestra una **limitación** del patrón: añadir un nuevo *tipo de producto* es más costoso que añadir una nueva *familia*.

Cuidado

Esta es una decisión de diseño importante: Abstract Factory brilla cuando se añaden **familias**, pero es costoso cuando se añaden **productos**. Tenlo en cuenta al elegir el patrón.

14. Paso 12: Integrar con otros patrones (inciso d)

Pregunta guía

El enunciado pide integrar con **Prototype** (para clonar personajes predefinidos) y **Singleton** (para el catálogo de fábricas). ¿Cómo lo harías?

Prototype: clonar personajes

```
        public abstract class PersonajePrototype : ICloneable
        {
            protected IArma arma;
            protected IArmadura armadura;
            protected IMontura montura;
            protected IHabilidad habilidad;

            public object Clone()
            {
                return this.MemberwiseClone();
            }
        }

```

Listing 10: Prototype para personajes

Razonamiento

En C#, el patrón Prototype se implementa con la interfaz `ICloneable` y el método `Clone()`. `MemberwiseClone()` hace una copia superficial (shallow copy).
El clonado permite crear personajes predefinidos (por ejemplo, “Guerrero Medieval”) y luego clonarlos para modificarlos sin afectar al original. Como el clon hereda las mismas referencias a productos, la consistencia de la era se mantiene.

Singleton: catálogo de fábricas

```
        public class CatalogoFabricas
        {
            private static CatalogoFabricas instancia;
            private Dictionary<string, IFactoryEra> fabricas;

            private CatalogoFabricas()
            {
                fabricas = new Dictionary<string, IFactoryEra>();
                fabricas["medieval"] = new FactoryMedieval();
                fabricas["futurista"] = new FactoryFuturista();
                fabricas["antigua"] = new FactoryAntigua();
            }

            public static CatalogoFabricas GetInstance()
            {
                if (instancia == null)
                {
                    instancia = new CatalogoFabricas();
                }
                return instancia;
            }

            public IFactoryEra GetFabrica(string era)
```

```
            {
                return fabricas[era];
            }
        }

```

Listing 11: Singleton para catálogo

Razonamiento

El Singleton garantiza que solo exista *un* catálogo de fábricas en toda la aplicación. Así, cualquier parte del código puede obtener una fábrica por nombre sin crear instancias repetidas.
En C#, se usa `Dictionary<string, IFactoryEra>` en lugar de `HashMap`. Para un Singleton thread-safe, se recomienda usar `Lazy<T>` o `lock`, pero para esta guía introductoria, la versión simple es suficiente.

15. Paso 13: Calcular el costo en clases (inciso e)

Pregunta guía

¿Cuántas clases tendrías si usarías herencia directa (una clase por cada combinación era-producto) en lugar de Abstract Factory?

Enfoque	Fórmula	Ejemplo (N=3, M=4)
Herencia directa	$N \times M$ clases de producto	12
Abstract Factory	$N \times M$ productos + N fábricas + M interfaces	$12 + 3 + 4 = 19$

Razonamiento

Abstract Factory tiene *más* clases, pero:

- El cliente no necesita conocer las $N \times M$ combinaciones.
- Añadir una era solo requiere $M + 1$ clases nuevas (productos + fábrica).
- La consistencia está garantizada por diseño.

Para N=10 eras y M=8 productos: herencia directa = 80 clases; Abstract Factory = 80 + 10 + 8 = 98 clases. Aunque hay más clases, el mantenimiento es mucho más sencillo.

16. Paso 14: Restricción final - Carga dinámica

Pregunta guía

El enunciado pide que las fábricas se carguen dinámicamente desde un archivo XML/JSON sin recompilar. ¿Cómo lo harías?

```
        using System;
        using System.Xml;
        using System.Reflection;
```

```
4
5       public class FabricaDinamica
6       {
7           public static IFactoryEra CargarDesdeXML(string ruta)
8           {
9               XmlDocument doc = new XmlDocument();
10              doc.Load(ruta);
11              string nombreClase = doc.GetElementsByTagName("fabrica")[0].
    InnerText;
12              Type tipo = Type.GetType(nombreClase);
13              return (IFactoryEra)Activator.CreateInstance(tipo);
14          }
15      }
16
```

Listing 12: Carga dinámica de fábricas

Razonamiento

En C#, la reflexión se hace con:

- `Type.GetType(nombreClase)` para obtener el tipo.
- `Activator.CreateInstance(tipo)` para crear la instancia.

Así, si mañana añaden una nueva era, solo hay que:

1. Compilar la nueva fábrica.
2. Añadir su nombre al XML.
3. Reiniciar la aplicación (sin recompilar el resto).

17. Resumen del proceso

Conclusión

Pasos que seguiste para resolver el problema:

1. Leer y comprender el problema.
2. Identificar los conceptos del dominio.
3. Reconocer el problema de diseño (solución ingenua y sus problemas).
4. Buscar un patrón adecuado (Abstract Factory).
5. Diseñar la estructura de clases.
6. Implementar productos abstractos.
7. Implementar productos concretos.
8. Implementar la fábrica abstracta.
9. Implementar las fábricas concretas.
10. Implementar el cliente.
11. Probar la solución.

12. Analizar la extensibilidad.
13. Integrar con Prototype y Singleton.
14. Calcular el costo en clases.
15. Cumplir la restricción de carga dinámica.

Este mismo proceso mental puedes aplicarlo a cualquier problema de diseño: **comprender, identificar, reconocer, buscar, diseñar, implementar, probar, analizar**.

18. Ejercicios propuestos

1. Añade una cuarta era (por ejemplo, "Steampunk") con sus 4 productos. ¿Cuántas clases nuevas creaste?
2. Añade un quinto tipo de producto (por ejemplo, `IEscudo`). ¿Qué clases tuviste que modificar?
3. ¿Qué pasaría si dos fábricas concretas compartieran un producto? Por ejemplo, que `FactoryMedieval` y `FactoryAntigua` usaran el mismo `Caballo`. ¿Es correcto?
4. Diseña un diagrama de secuencia que muestre cómo un `Personaje` crea sus productos a través de la fábrica.